

- Question 1 : Comment écrire la fonction d'échappement `hl7Escape(text)` pour les caractères réservés ?

○ Réponse (Code) :

JavaScript

```
function hl7Escape(text = "") {
  return String(text)
    .replace(/\\/g, '\\E\\')
    .replace(/\|/g, '\\F\\')
    .replace(/\^/g, '\\S\\')
    .replace(/~/g, '\\R\\')
    .replace(/&/g, '\\T\\');
}
```

- Question 2 : Comment implémenter la fonction de formatage temporel `ts(date)` au format `YYYYMMDDHHMMSS` ?

○ Réponse (Code) :

JavaScript

```
function ts(dateSource) {
  const d = new Date(dateSource);
  const pad = (n) => String(n).padStart(2, '0');
  return `${d.getFullYear()}${pad(d.getMonth() +
1)}${pad(d.getDate())}${pad(d.getHours())}${pad(d.getMinutes())}${pad(d.getSeconds())}`;
}
```

- Question 3 : Comment coder `generateHL7()` pour construire les segments `MSH` et `SCH` ?

○ Réponse (Code) :

JavaScript

```
function buildMSHandSCH(appointment) {
  const now = ts(new Date());
  const msh =
`MSH|^~\&|IMY-TRIFHIR|TELESUIVI|SIH|HOPITAL|${now}||SIU^S12|MSG${Date.now()}|P|
2.5`;
  const sch = `SCH|${hl7Escape(appointment.id)}|Booked`;
  return { msh, sch };
}
```

- Question 4 : Comment extraire les données pour générer le segment `PID` ?

- Réponse (Code) :

JavaScript

```
function buildPID(patient) {
  const id = patient.identifiant?.[0]?.value || patient.id || 'INCONNU';
  const family = hl7Escape(patient.name?.[0]?.family || '');
  const given = hl7Escape(patient.name?.[0]?.given?.[0] || '');
  const dob = patient.birthDate ? patient.birthDate.replace(/-/g, '') : '';
  const gender = patient.gender === 'male' ? 'M' : patient.gender === 'female' ? 'F' : 'O';

  return `PID|||${id}||${family}^${given}||${dob}|${gender}`;
}
```

- Question 5 : Comment générer les segments **RGS**, **AIS** et **AIP** ?

- Réponse (Code) :

JavaScript

```
function buildServicesAndPractitioner(appointment, practitioner) {
  const start = ts(appointment.start);
  const durationMin = Math.round((new Date(appointment.end) - new
Date(appointment.start)) / 60000);
  const docId = practitioner?.id || 'DOC-01';
  const docName = hl7Escape(practitioner?.name?.[0]?.family || 'Medecin');

  const rgs = `RGS|1`;
  const ais = `AIS|1||CONSULTATION|${start}|${durationMin}|min`;
  const aip = `AIP|1||${docId}^${docName}|${start}`;

  return [rgs, ais, aip].join("\r");
}
```

- Question 6 : Comment structurer la propriété calculée **hl7Mappings** pour la table de correspondance ?

- Réponse (Code) :

JavaScript

```
function getHl7Mappings(appointment, patient) {
  return [
    { fhirField: 'Appointment.id', hl7Target: 'SCH-1', value: appointment?.id },
    { fhirField: 'Appointment.status', hl7Target: 'SCH-25', value: appointment?.status },
    { fhirField: 'Appointment.start', hl7Target: 'AIS-4', value: appointment?.start },
    { fhirField: 'Patient.identifiant', hl7Target: 'PID-3', value: patient?.identifiant?.[0]?.value ||
patient?.id },
  ]
}
```

```

    { fhirField: 'Patient.name', hl7Target: 'PID-5', value: `${patient?.name?.[0]?.family}
    ${patient?.name?.[0]?.given?.[0]}` }
  ];
}

```

- Question 7 : Comment implémenter le tableau réactif **logs** et la fonction **log(method, url, status)** ?

- Réponse (Code) :

JavaScript

```
const logs = Vue.reactive([]);
```

```

function log(method, url, status, details = "") {
  logs.unshift({
    timestamp: new Date().toLocaleTimeString(),
    method,
    url,
    status,
    details
  });
  if (logs.length > 50) logs.pop();
}

```